Proving CGGI Bootstrapping with a Lattice Sumcheck Argument: an Expository Note

June 2026

Abstract

This note explains, at an introductory level, how a CGGI (TFHE) programmable bootstrapping is proven correct inside a lattice-based succinct argument. The construction has three parts. First, the bootstrapping itself is instantiated natively over the prime modulus $q = 2^{50} - 2687$ of the proof system, with the parameter shape of the TFHE-rs default set ($n = 918$ blind-rotation steps, ring degree $N = 2048$); one bootstrapping then runs in 0.3 seconds in software. Second, every polynomial of the execution trace is committed in a packed evaluation domain in which a product of TFHE-ring polynomials becomes a short convolution of products in the proof ring; this is what turns the 918 ciphertext multiplications of a blind rotation into constraints a sumcheck can process. Third, the whole trace is expressed as five sumcheck claims over one committed witness: linear claims for gadget recomposition and the public boundary values, copy claims that align multiplication operands, and one degree-two claim covering all multiplication steps at once, whose operands are *virtual*: recomposed from the committed short chunks inside the prover's sumcheck, at no extra commitment and no extra proof bytes. A bootstrapping at the full parameters is proven and verified end to end through the argument chain, including a random-projection round that binds a strict ℓ_2 norm of the committed trace: the prover runs in 2.5 minutes, the proof is 167 KB, and verification takes 12 seconds. At toy parameters, four independent bootstrappings under one committed key prove in 32 seconds and verify in 38 milliseconds, with the same proof size. The note assumes no prior familiarity with either TFHE or sumcheck arguments.

1 Introduction

Fully homomorphic encryption schemes of the CGGI family [1] evaluate functions on encrypted data by *bootstrapping*: a public-key operation that simultaneously refreshes the noise of a ciphertext and applies a lookup table to the encrypted value. A natural integrity question arises whenever an untrusted server performs bootstrappings: how can the server prove, succinctly, that the ciphertexts it returns really are the prescribed bootstrappings of the ciphertexts it received, under the key material it committed to?

This note describes such a proof, built on a lattice-based succinct argument [2] whose native objects are committed vectors over a cyclotomic ring and whose native statements are *sumcheck claims* about those vectors. The exposition is aimed at a reader who knows basic ring arithmetic but not necessarily TFHE or sumcheck protocols; every object is introduced before it is used.

The contributions described here are:

1. an implementation of CGGI bootstrapping directly over the proof system's prime modulus $q = 2^{50} - 2687$, with the parameter shape of the TFHE-rs 1.6.2 default set and noise rescaled accordingly (section 3);

2. a packing of TFHE-ring polynomials into proof-ring elements under which TFHE-ring multiplication becomes a length-16 convolution of native proof-ring multiplications (section 4);

3. an encoding of the complete blind-rotation trace, for several bootstrappings sharing one committed key, as five sumcheck claims over a single committed witness, with the multiplication operands recomposed *virtually* from their committed chunks inside the entry sumcheck (section 5);

4. an end-to-end instantiation at the full parameters: one bootstrapping proven in 2.5 minutes and verified in 12 seconds with a 167 KB proof, the committed trace bound in strict ℓ_2 norm by a random projection added to the entry round, and the same pipeline proving four toy-parameter bootstrappings in 32 s (section 6).

2 Background

2.1 CGGI bootstrapping in five objects

All ciphertexts live over $\mathbb{Z}_q$. We write p for the plaintext modulus (here $p = 32$, encoding a 2-bit message, a 2-bit carry, and a padding bit) and $\Delta = \lfloor q/p \rfloor$ for the encoding scale.

LWE. A Learning With Errors (LWE) ciphertext of a message m under a binary secret $\mathbf{s} \in \{0,1\}^n$ is a pair $(\mathbf{a}, b)$ with $\mathbf{a}$ uniform in $\mathbb{Z}_q^n$ and $b = \langle \mathbf{a}, \mathbf{s} \rangle + \Delta m + e$ for a small noise e. Decryption computes $b - \langle \mathbf{a}, \mathbf{s} \rangle$ and rounds to the nearest multiple of Δ.

GLWE. The ring variant (General LWE) replaces scalars by polynomials in $\mathcal{R}' = \mathbb{Z}_q[Y]/(Y^N + 1)$: a GLWE ciphertext under a key of k binary polynomials is a vector of $k+1$ polynomials, the first k uniform and the last carrying the message; we call its components *slots*, indexed $u = 0, \ldots, k$. Throughout, $k = 1$ and $N = 2048$, so a GLWE ciphertext is a pair of degree-2048 polynomials. Since $Y^N = -1$ in $\mathcal{R}'$, multiplying by the monomial Y^t rotates the coefficient vector by t positions with the wrapped-around coefficients re-entering negated, and Y^{-t} means Y^{2N-t}.

GGSW and the external product. A GGSW ciphertext (after the scheme of [3]) encrypts a single bit s_i in a redundant form: $(k+1)\ell$ GLWE rows, where row (u, j) carries $s_i B^j$ at slot u for a gadget base B and ℓ levels. Its purpose is the *external product*: given a GLWE ciphertext c, decompose each of its polynomials into ℓ *balanced* base-B digit polynomials (digit coefficients centered in $[-B/2, B/2)$ rather than $[0, B)$), multiply each digit polynomial by the matching GGSW row, and sum. The result is a GLWE encryption of $s_i \cdot$ (message of c), with noise that grew only proportionally to B rather than to q. The digit decomposition is the entire reason GGSW exists: multiplying c directly by an encryption of s_i would scale its noise by $q/2$ in the worst case.

CMUX. The external product, written $\boxdot$, gives an encrypted multiplexer:

$$\mathrm{CMUX}(\mathrm{GGSW}(s_i),\, d_0,\, d_1) \;=\; d_0 + \mathrm{GGSW}(s_i) \boxdot (d_1 - d_0),$$

which equals an encryption of the message of d_0 when $s_i = 0$ and of d_1 when $s_i = 1$, without revealing s_i.

Blind rotation. Bootstrapping an LWE ciphertext $(\mathbf{a}, b)$ proceeds by encoding the lookup table as a polynomial $L \in \mathcal{R}'$: the table has $p/2 = 16$ entries (the padding bit accounts for the sign ambiguity of negacyclic rotation), each repeated across a block of $N/16$ consecutive coefficients, with the block boundaries aligned so that the rounding error of the next step lands inside a block. The phase $b - \langle \mathbf{a}, \mathbf{s} \rangle$ is brought to the exponent of Y: a modulus switch rescales every component to $\tilde{a}_i = \lfloor a_i \cdot 2N/q \rceil$ and $\tilde{b} = \lfloor b \cdot 2N/q \rceil$, exponents in $\{0, \ldots, 2N-1\}$. The accumulator starts as $\mathrm{ACC} \leftarrow Y^{-\tilde{b}} \cdot L$ and runs, for $i = 1, \ldots, n$,

$$\mathrm{ACC} \;\leftarrow\; \mathrm{CMUX}\big(\mathrm{GGSW}(s_i),\; \mathrm{ACC},\; Y^{\tilde{a}_i} \cdot \mathrm{ACC}\big),$$

so that exactly when $s_i = 1$ the accumulator is rotated by $\tilde{a}_i$ positions. After all n steps the constant coefficient of the accumulator holds $\Delta \cdot L[m]$ up to small noise, and a public rearrangement of coefficients (*sample extraction*: the constant coefficient and the matching coefficients of the mask slot, read off as an LWE pair under the coefficient vector of the GLWE key) turns it back into an LWE ciphertext. Steps with $\tilde{a}_i = 0$ are skipped, since their CMUX is the identity; which steps survive is public, since $\mathbf{a}$ is public. We write $S \le n$ for the number of surviving steps (at the measured parameters, $S = 916$ of $n = 918$).

The blind rotation is the entire cost and the entire content of the proof: S CMUX steps, each containing one digit decomposition and $(k+1)^2\ell$ polynomial multiplications in $\mathcal{R}'$.

2.2 The proof system in four objects

The argument system [2] proves statements about a single committed vector. Four objects suffice to follow this note: the commitment, the multilinear extension, the sumcheck claim, and the argument chain.

Commitments. The prover arranges its data as a vector $\mathbf{w}$ of 2^ν elements of the proof ring $\mathcal{R} = \mathbb{Z}_q[X]/(X^{128}+1)$ and publishes its image under a public random linear compression, an Ajtai-style lattice commitment. Such a commitment is binding only for *short* vectors: producing two distinct short preimages of one image solves the Short Integer Solution problem, which is presumed hard, while long preimages are abundant and the map is far from injective on them. Every full-range value therefore enters $\mathbf{w}$ split into short chunks (section 5), and the argument must, alongside everything else, certify that the committed vector really is short.

Multilinear extensions. The multilinear extension of $\mathbf{w}$ is the unique polynomial $\mathrm{MLE}[\mathbf{w}]$ in ν variables, of degree at most one in each, that agrees with $\mathbf{w}$ on the Boolean cube: with $\mathrm{eq}(\mathbf{i}, \mathbf{z}) = \prod_{j=1}^{\nu} \big(i_j z_j + (1-i_j)(1-z_j)\big)$,

$$\mathrm{MLE}[\mathbf{w}](\mathbf{z}) \;=\; \sum_{\mathbf{i} \in \{0,1\}^\nu} \mathbf{w}[\mathbf{i}] \cdot \mathrm{eq}(\mathbf{i}, \mathbf{z}),$$

indexing $\mathbf{w}$ by the ν-bit representation of the position. At a Boolean point it reads one coordinate of $\mathbf{w}$; at a random point it compresses all of $\mathbf{w}$ into a single ring element. Sumchecks use both.

Sumcheck claims. The native statements of the system are claims of the form

$$\sum_{\mathbf{z}\in\{0,1\}^{\nu}} \sum_{\text{terms } t} \gamma_t \prod_{f\in t} f(\mathbf{z}) \;=\; v, \tag{1}$$

where each factor f, called an *oracle* below, is one of: MLE[$\mathbf{w}$] itself; the multilinear extension of a *segment* of $\mathbf{w}$ (fix the top μ index bits to a constant prefix; the $2^{\nu-\mu}$ entries under that prefix form a sub-vector whose MLE is a polynomial in the remaining $\nu-\mu$ low variables, constant in the rest); or the extension of a public vector. Since a product of oracles is evaluated at a common cube point $\mathbf{z}$, a degree-two term expresses a coordinatewise product of two committed values; section 5 adds two refinements to this factor vocabulary (a shifted segment and a virtual linear combination of segments) but no new power.

The sumcheck protocol [4] proves (1) in ν rounds. In round r the prover sends the univariate polynomial in z_r obtained by summing the claim's summand over the remaining $\nu-r$ Boolean variables (its degree is the largest number of oracles in a term); the verifier checks that its values at 0 and 1 add up to the running claim, and answers with a random challenge at which the running claim is re-evaluated. After ν rounds the original sum over 2^{ν} points has been reduced to the value of the summand at a single random point $\mathbf{c}$, which the verifier checks factor by factor: public oracles it evaluates itself, and each witness oracle becomes an *opening*, a claimed evaluation MLE[$\mathbf{w}$]($\mathbf{c}$) $= z$ that remains to be proven against the commitment. The prover's cost is linear in the vector length; the proof grows by ν small polynomials. Many claims are proven in one interaction by batching them with random weights into a single claim. This batched interaction over the claims of section 5 is called the *entry sumcheck* below.

The argument chain. What remains after the entry sumcheck is a batch of openings of one committed vector. The rest of the system, called the *chain* in this note, proves them by a sequence of fold-and-decompose rounds. The witness is laid out as a matrix; one round combines all its columns into one under transcript-derived short weights (a *fold*: linear, so the openings of the result are computable from the openings of the input), decomposes the folded column into balanced digits, reshapes the digits into the next, smaller matrix, commits to it, and proves consistency of the two commitments by a sumcheck. After enough rounds the remaining matrix is small enough to send in the clear. Along the way the chain certifies an aggregate ℓ_2 bound on the committed vector: the Euclidean norm of the integer coefficient vector obtained by concatenating the (centered) coefficients of all its ring elements. That bound is what keeps every commitment in its binding regime. The chain is taken from [2] unchanged, except for one reconfiguration described in section 6; its internals are not needed to follow this note. Each opening costs the chain one evaluation row, so the design below keeps the opening count small (it ends at 32).

Fiat–Shamir. All verifier randomness, in the entry sumcheck and in the chain, is derived by hashing the transcript of all prior prover messages (the Fiat–Shamir transform), so the proof is a single non-interactive string and the prover cannot steer the challenges. In particular, the random batching vectors ρ of section 5 are sampled from the hash state *after* the commitment to the witness is absorbed: the prover fixes its data before it learns the randomness that probes it.

Two consequences of this interface shape everything that follows. First, the claim language is *coordinatewise*: it multiplies committed values at equal positions, and offers no direct way

to multiply $\mathbf{w}[i]$ by $\mathbf{w}[j]$ for $i \neq j$. Second, the only smallness guarantee is an aggregate ℓ_2 norm; no per-coordinate range is proven. Both are deliberate economy choices of the underlying argument, and section 5 works within them.

3 Bootstrapping over the proof modulus

Standard TFHE works modulo 2^{64}. Proving statements about mod-2^{64} arithmetic inside a mod-q proof system would force every equation through an encoding of wraparounds. Instead, the bootstrapping is instantiated natively modulo the proof prime $q = 2^{50} - 2687$: the same algorithms, the same parameter shape, no second modulus anywhere.

Concretely, the implementation follows the default TFHE-rs 1.6.2 parameter set `PARAM_MESSAGE_2_CARRY_2` ($n = 918$, $k = 1$, $N = 2048$) with two adaptations. Noise bounds scale by $q/2^{64}$: the LWE noise is uniform on $[-2^{31}, 2^{31}]$ and the GLWE noise on $[-2^3, 2^3]$, preserving the noise-to-modulus ratios of the original set. The gadget changes from the base-2^{23}, one-level decomposition of TFHE-rs, which relies on power-of-two bit extraction, to an exact balanced decomposition with $B = 2^{25}$ and $\ell = 2$, natural for a prime modulus since $B^\ell = 2^{50} \approx q$ lets two balanced digits represent every residue exactly; an exact decomposition is a linear identity, with no rounding remainder to carry through the constraints. The modulus switch, the redundant lookup table layout, the skip of zero steps, and the key-switching follow the reference implementation.

Polynomial multiplication in $\mathcal{R}'$ uses a partial negacyclic NTT (number-theoretic transform: the evaluation-domain representation in which polynomial multiplication is coordinatewise). The 2-adic valuation of $q - 1$ is 7, so roots of unity of order at most 2^7 exist modulo q and a full NTT at $N = 2048$ does not; six transform levels split $Y^{2048} + 1$ into 64 blocks of degree 32, and the blockwise products run on AVX-512 IFMA kernels (size-64 NTT columns and 52-bit multiply-accumulate convolution, with a final fold of high bits by 2687, using $2^{50} \equiv 2687 \pmod q$). One multiplication takes 26 μs, and a complete bootstrapping (mod switch, 918 CMUX steps, sample extraction) runs in 0.3 s single-threaded; generating the full key material and one traced bootstrapping takes 0.7 s. Correctness is tested end to end: all 16 plaintexts bootstrap through a nontrivial lookup table and decrypt correctly, both after sample extraction and after key-switching back to the small key.

4 Packing the TFHE ring into the proof ring

The proof system multiplies elements of $\mathcal{R}$, of degree 128; the trace multiplies elements of $\mathcal{R}'$, of degree 2048. The bridge is a factorization both rings share. Since the 2-adic valuation of $q - 1$ is 7, the group $\mathbb{Z}_q^*$ contains elements of order 128, hence 64 roots of $\psi^{64} = -1$; modulo q the polynomial $X^{128} + 1$ splits into the 64 quadratic factors $X^2 - \psi_j$, and by the Chinese remainder theorem an element of $\mathcal{R}$ is the same thing as its 64 residues modulo these factors, which we call its *quadratic slots*. Multiplication in $\mathcal{R}$ acts independently on the slots (this is the incomplete NTT the proof system computes with). The same ψ_j govern the TFHE ring:

$$\mathbb{Z}_q[Y]/(Y^{2048} + 1) \;\cong\; \prod_{j=1}^{64} \mathbb{Z}_q[Y]/(Y^{32} - \psi_j) \;\cong\; \prod_{j=1}^{64} \mathbb{F}_{q^2}[Y]/(Y^{16} - x_j),$$

where the first step is again the Chinese remainder theorem ($Y^{2048}+1=\prod_j(Y^{32}-\psi_j)$ for the same ψ_j), and the second step writes each degree-32 residue over the quadratic extension field $\mathbb{F}_{q^2}=\mathbb{Z}_q[X]/(X^2-\psi_j)$, with x_j the class of X there (so $x_j^2=\psi_j$) and $Y^{16}\mapsto x_j$.

A polynomial $P\in\mathcal{R}'$ is therefore stored as 16 elements of $\mathcal{R}$, written $P_0,\dots,P_{15}$: element P_t holds, in its j-th quadratic slot, the Y^t-coordinate of the j-th residue of P. Under this packing, a product in $\mathcal{R}'$ becomes

$$(P\cdot R)_t \;=\; \sum_{t_1+t_2=t} P_{t_1}\odot R_{t_2} \;+\; X\odot \sum_{t_1+t_2=t+16} P_{t_1}\odot R_{t_2}, \tag{2}$$

where $\odot$ is multiplication in $\mathcal{R}$, the first sum ranges over $t_1+t_2=t$ and the second over the pairs that overshoot the period ($t_1+t_2=t+16$ with $t_1,t_2\le 15$); the overshooting terms carry one factor $Y^{16}=x_j$ per slot, which is exactly one multiplication by the public element $X\in\mathcal{R}$. One TFHE-ring product is thus 256 proof-ring products arranged as a length-16 convolution. The packing and identity (2) are verified in code against direct multiplication in $\mathcal{R}'$ for $N\in\{256,2048\}$.

The point of working in this packed evaluation domain, rather than committing coefficient vectors, is that (2) contains only *ring products of stored values*: the claim language of section 2.2 can express it. A coefficient-domain encoding would instead turn each polynomial product into a dense 2048×2048 linear map interleaved with products, far outside coordinatewise reach.

5 The relation

Fix a bootstrapping and consider its surviving steps $1,\dots,S$. Indices are used as follows: $u,v\in\{0,\dots,k\}$ are GLWE slots (u for the decomposed input slot, v for the output slot), $j\in\{0,\dots,\ell-1\}$ is the gadget level, and $t\in\{0,\dots,15\}$ is the packing coordinate of section 4. In the CMUX of step i, the external product decomposes the rotated difference: writing $c'=Y^{\tilde{a}_i}\mathrm{ACC}_{i-1}-\mathrm{ACC}_{i-1}$, slot u of c' is decomposed into digit polynomials $D_{u,j}$, and slot v of the product is $\sum_{u,j} D_{u,j}\cdot G_{u,j,v}$, where $G_{u,j,v}$ denotes slot v of GGSW row (u,j).

The committed witness contains, in the packed representation of section 4:

- the accumulator states $\mathrm{ACC}_0,\dots,\mathrm{ACC}_S$ ($k+1=2$ polynomials each);
- for each step, the digit polynomials $D_{u,j}$;
- the GGSW rows $G_{u,j,v}$ of the bootstrapping key, stored once and shared by all bootstrappings in the batch;
- two *operand lists*, described below.

Packed values fill the full range of $\mathbb{Z}_q$, while the commitment is binding only for short vectors, so every stored coordinate is decomposed into eight balanced chunks of 7 bits, the chunks of one quantity forming eight parallel *planes* of the witness; a value is recovered as $\sum_c 2^{7c}(\text{chunk } c)$, and all claims act on the planes through this linear recomposition. The chunk width is the same base-2^7, eight-digit shape the argument chain uses internally for every full-range vector it commits, and for the same reasons: the digits set the norm of the committed vector, and the commitment's binding margin degrades as that norm grows; the chain's first fold-and-decompose step combines all witness columns within a fixed per-coefficient capacity; and the entry-round random projection of section 6 accumulates digits in 16-bit lanes, which stay in range only for

digits at this scale. An earlier iteration used four 15-bit chunks, half the storage, and tripped every one of these constraints in turn.

The blind-rotation step relation splits into one linear and one quadratic family.

Recomposition and boundaries (linear). For each step and each slot coordinate, the digits must recompose to the rotated difference:

$$\sum_j B^j D_{u,j} \;=\; Y^{\tilde{a}_i}\mathrm{ACC}_{i-1} - \mathrm{ACC}_{i-1},$$

where multiplication by the public monomial $Y^{\tilde{a}_i}$ is, in the packed domain, a public length-16 convolution: a linear map with known coefficients. Together with the boundary equalities, $\mathrm{ACC}_0 = \mathrm{pack}(Y^{-\tilde{b}}L)$ and $\mathrm{ACC}_S =$ the public output accumulator, these are all linear equations in the witness. They are batched into a single claim of the form (1): a random vector ρ, sampled from the transcript after the commitment, weights every equation, and the per-position weights are assembled into one public vector per witness segment. The claim value collects the ρ-weighted public boundary data.

Accumulation (quadratic). For each step and each output slot v, the accumulator must advance by the external product:

$$\mathrm{ACC}_{i,v} \;=\; \mathrm{ACC}_{i-1,v} + \sum_{u,j} D_{u,j} \cdot G_{u,j,v},$$

with the products expanded by (2) into pairs $D_{u,j,t_1} \odot G_{u,j,v,t_2}$. Two obstacles stand between these pairs and the claim language of section 2.2: the pairs multiply witness values at *different* positions, while claims multiply values at equal positions only; and each operand is stored as eight chunk planes, while the equation needs the full-range value.

The first obstacle is removed by aligning the operands on a common index space. Write g for the multiplication group: one value of g per triple (step i, slot u, level j), so $S(k+1)\ell$ groups per bootstrapping. Order the product space by (t_1, g, t_2). The witness carries two operand lists: lop, indexed by (t_1, g) and holding the digit coordinate D_{g,t_1}, and one rop_v per output slot, indexed by (g, t_2) and holding the key coordinate G_{g,v,t_2}. Each list has exactly one entry per distinct operand; nothing is replicated. Inside the entry sumcheck, rop_v is read as an ordinary segment oracle over the low (g, t_2) variables, and lop as a *shifted* oracle: the same segment construction, but with its data variables identified with the (t_1, g) block of the index space, constant in the low t_2 block. A product of the two at a cube point (t_1, g, t_2) is then exactly $D_{g,t_1} \odot G_{g,v,t_2}$. The shifted oracle's evaluation enters the chain as one more opening, at the point whose coordinates are the (t_1, g)-slice of the sumcheck's challenge point.

The second obstacle is dissolved rather than paid for: the recomposed operands are *virtual*, existing only inside the prover's sumcheck, never in the witness. The factor vocabulary of (1) is extended by one entry: a public linear combination of segment oracles over a shared index layout, here

$$\textstyle\sum_{c=0}^{7} 2^{7c} \cdot \mathrm{MLE}[\mathrm{plane}_c]$$

for the eight chunk planes of an operand list. The prover materializes the combination once, as a single vector the size of one plane, and runs its sumcheck rounds on it like on any other oracle. No new commitment and no new opening appear: each plane is already opened at the entry sumcheck's evaluation point on behalf of the copy claims below, and by linearity the virtual

oracle's evaluation equals the same 2^{7c}-weighted sum of those plane openings, so the verifier reconstructs it from values the chain certifies anyway. The alternative, distributing the chunks of both operands across the claim, costs 8×8 quadratic terms per output slot where the virtual operands cost one; the factor is visible directly in the entry sumcheck times of section 6. All accumulation equations therefore batch into a single claim with one degree-two term per output slot: a public weight vector, carrying the transcript-derived equation weights and the X-twist of (2), multiplies the virtual recomposed digit operand and the virtual recomposed key operand; linear terms carry the ACC sides of the equations.

Copy claims. It remains to tie the operand lists to their sources. Write o for a position in an operand list, e for a position in the digit (respectively key) planes, and $\mathrm{src}(o)$ for the plane position that list entry o is supposed to copy. A transcript-derived random vector ρ satisfies $\sum_o \rho_o \, \mathsf{lop}(o) = \sum_e \big(\sum_{o:\mathrm{src}(o)=e} \rho_o\big) D(e)$ whenever the list is a correct rearrangement, and equality of the two random linear combinations, checked plane by plane within one claim, binds the list to its source planes. This yields three copy claims: one for lop and one for each of the $k+1=2$ lists rop_v; together with the linear claim and the accumulation claim, five claims in total. Since the lists are permutation-sized, the operand storage equals one extra copy of the digits and $k+1$ rearrangements of the used key rows.

What the proof asserts. The claims above, plus the ℓ_2 bound certified by the chain, prove knowledge of accumulator states, digits, and key rows that satisfy every step equation, recompose correctly, have bounded aggregate norm, and connect the public input phase to the public output accumulator. The norm bound is doing real work in this statement. The recomposition equations alone do not pin the digits down: every difference $Y^{\tilde{a}_i}\mathrm{ACC}_{i-1} - \mathrm{ACC}_{i-1}$ admits infinitely many decompositions into ℓ summands with the right scales, almost all of them with huge coordinates, and an external product against huge digits encrypts the right value with unboundedly large noise. What makes the committed digits behave like gadget digits is only their smallness, and the proof binds that smallness in aggregate: the strict ℓ_2 bound on the whole trace, certified by the entry projection of section 6, bounds each digit coordinate by the full trace budget (a single coordinate may spend the whole budget) and hence caps the noise a dishonest decomposition can inject. Per-coordinate ranges are not proven; a prover may substitute any decomposition of the correct differences whose mass fits the ℓ_2 budget, so the statement is *knowledge of a bounded-norm execution* rather than of the canonical one. For the output ciphertext to be usable, the parameter analysis must therefore budget noise against the certified norm bound rather than against $B/2$; tightening the statement to the canonical trace would require range claims on the digits.

Design decisions. The encoding embodies a series of choices, each trading generality for prover economy; they are collected here with the alternative each one displaced.

- *Native modulus* (section 3). The bootstrapping runs over the proof prime q itself. The alternative, proving the standard mod-2^{64} execution inside the mod-q system, would add a committed wraparound count to every addition and multiplication, each needing its own smallness claim.
- *Exact gadget* (section 3). The balanced base-2^{25}, two-level decomposition replaces the rounded one-level gadget of TFHE-rs. A rounded gadget leaves a rounding remainder in

every external product: one more committed quantity per product, with its own smallness obligation. An exact decomposition is a linear identity and costs the relation nothing beyond the digits themselves.

- *Evaluation-domain packing* (section 4). Committing packed residues makes a TFHE-ring product a short convolution of coordinatewise products. Committing coefficient vectors would make it a dense 2048×2048 linear map interleaved with the products, outside the claim language.

- *Digit smallness by norm, not by range.* The gadget digits are bound only through the strict aggregate ℓ_2 norm of the committed trace, as discussed above. Per-coordinate range claims would need, for each of the 7.5 million digit coordinates ($S = 916$ steps, $(k+1)\ell = 4$ digit polynomials each, 2048 coefficients), either a degree-B product constraint or a bit decomposition of the coordinate; both multiply the witness and the claim degree. The aggregate norm is what the argument certifies natively, and accepting it as the smallness statement is what keeps the relation at five claims.

- *Chunk shape mirrors the chain* (section 5). Eight balanced 7-bit chunks per coordinate, the same digit geometry the chain uses internally, keeping the committed norm, the fold capacity, and the 16-bit projection lanes all in their working ranges.

- *Operands deduplicated, recomposed virtually* (section 5). The operand lists hold each distinct operand once and are read through shifted oracles; the full-range operands exist only as virtual linear combinations inside the prover. The alternatives, replicating operands across the 16×16 product positions or committing the recomposed values next to their chunks, multiply the committed volume and the opening count.

- *Strict norm via the entry projection* (section 6). The chain's polynomial-commitment mode tolerates a slack factor on the extracted norm; here the norm is the security statement about the digits, so the entry round pays for a random projection of the trace and the chain runs at doubled interior heights to absorb its image.

- *One key per batch* (section 5). The bootstrapping key is committed once and shared by all bootstrappings in the batch, so its cost amortizes; the copy claims that route key rows to multiplication groups are what make the sharing possible.

6 End-to-end instantiation

The complete pipeline commits the witness, runs the entry sumcheck over the five claims of section 5 (one linear, three copy, one quadratic), and hands the resulting evaluation claims to the argument chain of [2] as its initial openings: the two standard witness evaluations plus one opening per witness segment, twenty-five segments here (the stacked linear segment holding accumulators, digits, and key; the eight chunk planes of lop; and the $8 \times (k+1) = 16$ chunk planes of the two rop_v), padded to thirty-two. The virtual recomposed operands add no openings.

One component of the chain is reconfigured rather than reused as-is. In its polynomial-commitment role the chain skips the random norm projection in the entry round, because there the entry witness consists of decomposition digits whose shortness the chain re-derives downstream; the extracted norm bound then carries a slack factor, which that application tolerates.

	toy, 4 bootstrappings	full, 1 bootstrapping
TFHE parameters	$N = 256$, $n = 8$	$N = 2048$, $n = 918$
committed witness (ring elements)	2^{20}	2^{22}
key generation and traced execution	< 0.01 s	0.7 s
witness encoding	0.3 s	9.1 s
commitment	1.0 s	3.4 s
entry sumcheck	20.4 s	109.1 s
chain rounds	10.4 s	38.7 s
prover, commitment through chain	31.7 s	151.2 s
proof size	166.4 KB	166.6 KB
verification	0.038 s	11.7 s

Table 1: End-to-end measurements, single-threaded on one Intel i7-11850H core.

Here the witness is the trace itself, and the ℓ_2 statement should bind it directly. The entry round therefore runs the chain's coarse projection on the committed trace: a transcript-sampled random ± 1 block matrix is applied to the witness coordinates (balanced 7-bit chunks, so the blockwise sums fit comfortably in the implementation's signed 16-bit accumulator lanes), the image is committed alongside the other entry data, and the consistency of image and witness is checked by the next round's sumcheck, exactly as in the chain's interior rounds. The image does not come for free: it is 1/64 of the witness in elements and, after its own digit decomposition, 1/16 in committed volume, which exceeds the headroom of the standard chain geometry. Every interior round of the chain therefore runs with an enlarged witness matrix (the first three rounds at four times the standard number of rows, the last three at twice), and the final fold is reshaped to hand the last round its usual input; the last round itself is unchanged. The configuration generator now checks each round's composed output against the next round's capacity, a mismatch that previously failed only as a debug assertion.

Table 1 reports the complete pipeline at both parameter scales. At the toy parameters ($N = 256$, $n = 8$) four independent bootstrappings under one committed key fill a 2^{20}-element witness; at the full parameters of section 3 the trace of one bootstrapping occupies about 3.7 million committed ring elements, filling the 2^{22} middle parameter set of [2], and two bootstrappings fit the largest set (2^{24}). At both scales the entry sumcheck dominates the prover, and within it the dense public weight vectors of the batched claims. Verification at the full parameters is dominated not by the chain, which takes milliseconds, but by re-deriving and evaluating the dense Fiat–Shamir public vectors of the entry claims (the assembled ρ-weight vectors of section 5), linear in their combined length.

Correctness of every layer is tested independently: the packing against direct TFHE-ring multiplication, the recorded traces against step-by-step replay, each claim family in isolation, every entry-sumcheck opening against direct evaluation of the committed witness, the four-bootstrapping batch, and rejection of a tampered output accumulator.

7 Limitations

Five gaps separate this instantiation from a production verifiable-bootstrapping service. The digit ranges are bounded only in aggregate ℓ_2 norm, as discussed in section 5; the entry-round

projection of section 6 makes that aggregate bound strict, but does not turn it into per-coordinate ranges. The proven relation covers the blind rotation, which is the only place the committed key material enters; the modulus switch, sample extraction, and key switch act on public data with public keys, so the verifier can recheck them directly. What is not proven is any well-formedness of the committed bootstrapping key itself, or its consistency with the public key-switching key: the statement is relative to the committed key material as given. The final accumulator is published in full, which reveals N valid LWE ciphertexts rather than the single extracted one; harmless under the scheme's own security but coarser than necessary. The verifier re-derives the entry round's Fiat–Shamir public vectors explicitly and evaluates them coordinate by coordinate; they are highly structured (scattered batching randomness and equality weights), so a succinct verifier should evaluate them lazily from that structure rather than materialize them, and until it does, verification time at full parameters is linear in the trace. And the security of the rescaled parameter set, while inheriting the noise ratios of the TFHE-rs original, has not been re-estimated against lattice attacks at the new modulus; the parameter-selection pass for both the FHE side and the argument side remains future work. Finally, one digit window of the chain operates close to capacity: at the full parameters the largest coefficient of the round-one fold reaches 90% of the range representable by that round's two base-2^6 digits, and since the Fiat–Shamir challenges are a deterministic function of the instance, an instance whose fold exceeds the window cannot be proven under the current chain geometry (a debug build detects the event; none occurred in the reported runs). Widening the window is a rebalance of the chain's digit cascade, also left as future work.

References

[1] I. Chillotti, N. Gama, M. Georgieva, M. Izabachène. TFHE: Fast Fully Homomorphic Encryption over the Torus. Journal of Cryptology, 2020. Implementation: TFHE-rs 1.6.2, https://github.com/zama-ai/tfhe-rs.

[2] RoKoko: Lattice-based Succinct Arguments, a Committed Refinement. 2026. Implementation: https://github.com/lattice-arguments/rokoko.

[3] C. Gentry, A. Sahai, B. Waters. Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. CRYPTO 2013.

[4] C. Lund, L. Fortnow, H. Karloff, N. Nisan. Algebraic Methods for Interactive Proof Systems. Journal of the ACM, 39(4), 1992.